UNIT 1 · TOPIC 1.5

Leveraging AI in Cyber Defense

Assignment, Answer Key

Teacher answer key, 39 points total.

Practice Questions

1.5.a · AI reviewing security configurations

1. A config file contains 'PermitRootLogin yes'. What is the DIRECT security risk? (1 pt)

- ○ A. The SSH port becomes publicly visible
- **● B. An attacker who cracks the root password has immediate full system access**
- ○ C. The system disables all password authentication
- ○ D. TLS encryption is bypassed

Why: PermitRootLogin yes allows the root account to authenticate directly over SSH. Root compromise = total system control with no need to escalate privileges.

2. Which of the following settings in an SSH config file would an AI auditor flag as insecure? Select ALL that apply. (3 pts)

- **■ A. PasswordAuthentication yes**
- □ B. Port 22
- **■ C. PermitRootLogin yes**
- **■ D. X11Forwarding yes**

Why: PasswordAuthentication yes enables brute-force attacks; PermitRootLogin yes allows direct root access; X11Forwarding yes exposes unnecessary attack surface. Port 22 is the standard SSH port and is not itself insecure.

3. Why does an AI config auditor emit a FINDING rather than automatically changing the insecure setting? (1 pt)

- ○ A. AI tools cannot read configuration files
- ○ B. Configuration files are encrypted
- **● C. The setting may have a compensating control or documented exception that the human reviewer must evaluate**
- ○ D. Audit findings are only used for compliance reports, not remediation

Why: A finding is a question, not a verdict. The human reviews whether a compensating control exists or the exception is documented. The AI handles mechanical pattern matching; the engineer provides context.

4. A config line reads 'port 23'. Why does an AI auditor flag this? (1 pt)

- ○ A. Port 23 is reserved and cannot be opened
- **● B. Port 23 is Telnet, which transmits credentials in cleartext**
- ○ C. Ports below 1024 are always insecure
- ○ D. Port 23 conflicts with HTTP traffic on port 80

Why: Port 23 is the Telnet protocol, which sends all data including passwords unencrypted. SSH (port 22) replaced it specifically because of this vulnerability.

5. Name the security lifecycle stage where config review runs inside a CI/CD pipeline to block insecure settings BEFORE they reach production. (1 pt)

Acceptable answers: shift-left; shift left; pre-deployment; pre deployment; before deployment

Why: Shift-left security means moving checks earlier in the development cycle. Running a config auditor on a pull request catches misconfigurations before any server is affected.

1.5.b · AI analyzing code for vulnerabilities

6. Why is `eval(user_input)` flagged as dangerous by a static code scanner? (1 pt)

○ A. eval() is deprecated and no longer part of Python

● **B. eval() executes an arbitrary string as Python code, letting an attacker run any code on the server**

○ C. eval() only works with numbers and cannot process strings

○ D. eval() is slow and causes performance issues

Why: eval() interprets its argument as Python code. If user-controlled input reaches eval(), the attacker controls what code runs, a classic code injection vulnerability.

7. Which of the following Python patterns would an AI code scanner flag as causing OS command injection risk? Select ALL that apply. (2 pts)

■ **A. os.system(cmd)**

□ B. subprocess.run(args, shell=False)

■ **C. subprocess.run(cmd, shell=True)**

□ D. os.path.join(base, filename)

Why: os.system() always passes commands to a shell. subprocess with shell=True restores the same injection risk. subprocess with shell=False (a list of args) is safe. os.path.join is not a command execution function.

8. A developer stores `password = 'Admin@2024'` directly in source code. What is the PRIMARY security risk? (1 pt)

○ A. The password is too short

○ B. The equals sign is an insecure operator

○ C. Python cannot store passwords in variables

● **D. The password is visible to anyone with access to the source code or version history, making it a leaked credential**

Why: Hardcoded credentials are embedded in source code checked into version control. Anyone who can read the repo, including attackers who breach the code host, can see the credential.

9. Why is `pickle.loads(data)` dangerous when `data` comes from an untrusted source? (1 pt)

○ A. pickle.loads() is too slow for production use

● **B. Deserialization of a maliciously crafted pickle can execute arbitrary code without any additional exploit**

○ C. pickle.loads() only works with binary data

○ D. pickle.loads() is not available in Python 3

Why: Pickle deserialization executes Python code embedded in the serialized object. An attacker can craft a malicious pickle that runs any command when loaded, making pickle from untrusted input equivalent to code execution.

10. Name the key difference between STATIC analysis and DYNAMIC analysis of source code. (1 pt)

Acceptable answers: static does not run; without executing; without running; examines without execution; no execution

Why: Static analysis reads code without executing it. Dynamic analysis runs the code and observes behavior. Static is fast and catches pattern-based bugs; dynamic finds runtime-dependent bugs.

1.5.c · AI suggesting detection rules

11. A detection rule fires when the same source IP generates 5 failed logins within 60 seconds. What is the name for the numeric limit (5) in this rule? (1 pt)

- ○ A. Signature
- ○ B. Baseline
- **● C. Threshold**
- ○ D. Precision

Why: The minimum event count required to trigger a rule is called the threshold. Tuning it balances false positives against missed detections.

12. An analyst sets the brute-force threshold to 1 failed login. What is the MOST LIKELY consequence? (1 pt)

- ○ A. Attackers will never be detected
- ○ B. The SIEM will run out of storage
- **● C. Alert fatigue -- analysts are overwhelmed by normal mistyped passwords**
- ○ D. The network will block all traffic

Why: A threshold of 1 flags every mistyped password, generating far more alerts than analysts can investigate -- a classic alert fatigue scenario.

13. Which of the following are components of a well-formed detection rule? Select ALL that apply. (3 pts)

- **■ A. A condition (what pattern to match)**
- **■ B. A threshold (how many events)**
- **■ C. An action (what to do when triggered)**
- □ D. A list of attacker names

Why: Rules have three parts: condition, threshold, and action. Attacker names are not part of the rule structure.

14. What does SIEM stand for, and what is its primary function in a SOC? (1 pt)

Acceptable answers: security information and event management; aggregate logs; correlate logs; apply detection rules

Why: SIEM = Security Information and Event Management. It aggregates logs from many sources, applies detection rules, and surfaces alerts to analysts.

15. An AI tool proposes a new detection rule based on historical log analysis. What should happen BEFORE the rule is deployed to production? (1 pt)

- ○ A. Deploy it immediately -- AI rules are always correct
- ○ B. Delete it -- AI-generated rules are unreliable
- ○ C. Run it for one second and check if any alerts fire
- **● D. A human analyst reviews it for precision, context, and compliance**

Why: AI generates rule candidates; humans approve. Analyst review checks precision, legal/privacy compliance, and whether the IP might be a trusted partner.

1.5.d · Faster, more accurate threat detection and response

16. MTTR stands for Mean Time to Respond. Which formula correctly defines it for a set of N incidents? (1 pt)

- **● A. The sum of all response times divided by the number of incidents**
- ○ B. The time the fastest incident took to resolve
- ○ C. The number of auto-resolved incidents divided by total incidents
- ○ D. The longest response time in the dataset

Why: MTTR = sum(resolved_at - detected_at) / N. It is the arithmetic mean of individual response times across all incidents.

17. Which type of incident is MOST appropriate for automated (no-human) resolution? (1 pt)

○ A. A novel ransomware variant with no known decryption key

● **B. A high-confidence known-malicious IP blocked at the firewall with a reversible action**

○ C. An alert involving a critical business partner's IP address

○ D. An incident requiring communication with law enforcement

Why: Auto-resolution is safe only for high-confidence, reversible actions. Novel patterns, business-partner IPs, and law-enforcement cases require human judgment.

18. AI-assisted triage scoring considers which of the following signals? Select ALL that apply. (3 pts)

■ **A. Type of event (e.g., ransomware behavior vs. single failed login)**

■ **B. Criticality of the affected asset (e.g., domain controller vs. developer laptop)**

□ C. The analyst's personal preference for ticket order

■ **D. Historical false-positive rate for similar alert types**

Why: AI triage weighs event type, asset criticality, threat-intelligence reputation, and historical FP rates. Analyst preference is not an AI signal.

19. Name ONE sub-component of MTTR that AI improves by continuously monitoring telemetry without attention lapses. (1 pt)

Acceptable answers: time to detect; ttd; detection time

Why: TTD (Time to Detect) shrinks because AI monitors telemetry 24/7 without fatigue, catching events that a tired analyst might miss during overnight shifts.

20. A SOC processes incidents in arrival order instead of severity order. What is the PRIMARY risk? (1 pt)

○ A. The SIEM will generate fewer alerts

○ B. Auto-resolution will stop working

● **C. Critical incidents may wait behind low-severity noise, giving attackers more dwell time**

○ D. MTTR will become negative

Why: Processing in arrival order means a critical ransomware alert can sit behind hundreds of low-severity port-scan alerts, increasing dwell time and damage.

Coding Assignments (Python 3)

1.5.a · AI reviewing security configurations

3 pts · auto-graded on the site

Coding problem, 1.5.a: AI reviewing security configurations

Skill: Mitigate Risk, Collaborate · **Maps to:** 1.5.A · **Language:** Python 3 (pytest)

Problem

Build the rule-engine core of an AI-assisted configuration auditor. Given a list of configuration file lines (as a real tool would read from `/etc/ssh/sshd_config` or a service config), return the sorted set of security finding tags for every insecure directive detected. This mirrors exactly what tools like Lynis and OpenSCAP do at their rule-evaluation layer.

Function contract

Implement `solve(config_lines: list[str]) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return a sorted, de-duplicated list of finding tag strings.

Input

A `list[str]`, each element one line from a config file. Lines may be blank, start with # (comments), or contain leading/trailing whitespace. Matching is case-insensitive.

Example input:

```
[
    "# sshd_config for web01",
    "Port 22",
    "PermitRootLogin yes",
    "PasswordAuthentication yes",
    "X11Forwarding yes",
    "",
]
```

Output

A sorted `list[str]` of finding tags. Tags are drawn from:

- `"insecure:PermitRootLogin"`
- `"insecure:PasswordAuthentication"`
- `"insecure:Port23"`
- `"insecure:SSLDisabled"`
- `"insecure:X11Forwarding"`
- `"insecure:PermitEmptyPasswords"`

Examples

```
["PermitRootLogin yes", "Port 22"]
    -> ["insecure:PermitRootLogin"]

["PasswordAuthentication yes", "X11Forwarding yes", "PermitRootLogin yes"]
    -> ["insecure:PasswordAuthentication", "insecure:PermitRootLogin", "insecure:X11Forwarding"]

["port 23", "ssl false"]
    -> ["insecure:Port23", "insecure:SSLDisabled"]

["# all defaults", "Port 22", "PermitRootLogin no", ""]
    -> []
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 1.5.a. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
from typing import List

# Rule set: each entry is (tag, list_of_trigger_substrings).
# A line triggers the tag if ANY trigger substring matches (case-insensitive).
RULES = [
    ("insecure:PermitRootLogin",      ["permitrootlogin yes"]),
    ("insecure:PasswordAuthentication", ["passwordauthentication yes"]),
    ("insecure:Port23",               ["port 23"]),
    ("insecure:SSLDisabled",          ["ssl false", "ssl off", "tls false", "tls off"]),
    ("insecure:X11Forwarding",        ["x11forwarding yes"]),
    ("insecure:PermitEmptyPasswords", ["permitemptypasswords yes"]),
]


def solve(config_lines: List[str]) -> List[str]:
    """Return sorted, de-duplicated security finding tags for insecure config directives."""
    findings = set()
    for raw in config_lines:
        line = raw.strip()
        if not line or line.startswith("#"):
            continue
        low = line.lower()
        for tag, triggers in RULES:
            if any(t in low for t in triggers):
                findings.add(tag)
    return sorted(findings)
```

1.5.b · AI analyzing code for vulnerabilities

3 pts · auto-graded on the site

Coding problem, 1.5.b: AI analyzing code for vulnerabilities

Skill: Detect Attacks, Collaborate · **Maps to:** 1.5.A · **Language:** Python 3 (pytest)

Problem

Build the pattern-matching core of an AI code security scanner. Given a string of Python source code, return a sorted list of vulnerability tags for every dangerous pattern found. This is the rule-evaluation layer that tools like Bandit and Semgrep implement under the hood.

Function contract

Implement `solve(code: str) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`). Return a sorted, de-duplicated list of finding tag strings.

Input

A single string, the entire source code to scan. May be multi-line or empty.

Output

A sorted `list[str]` of vulnerability tags drawn from:

- `"vuln:eval"`
- `"vuln:hardcoded_password"`
- `"vuln:md5"`

- "vuln:os_system"
- "vuln:pickle_loads"
- "vuln:subprocess_shell"

Examples

```
'result = eval(user_input)'
    -> ["vuln:eval"]

'os.system("ls " + path)\nsubprocess.run(cmd, shell=True)'
    -> ["vuln:os_system", "vuln:subprocess_shell"]

'import hashlib\nhashlib.md5(data)\npickle.loads(blob)'
    -> ["vuln:md5", "vuln:pickle_loads"]

'x = 1 + 2'
    -> []
```

Constraints

- Deterministic, offline, standard library only.
- `password` = matching is case-insensitive; all other patterns are case-sensitive.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 1.5.b. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml)."""
from typing import List

# Each rule: (tag, trigger_string, case_sensitive)
RULES = [
    ("vuln:eval",               "eval(",        True),
    ("vuln:hardcoded_password", "password =",   False),  # case-insensitive
    ("vuln:md5",                "md5(",         True),
    ("vuln:os_system",          "os.system(",   True),
    ("vuln:pickle_loads",       "pickle.loads(", True),
    ("vuln:subprocess_shell",   "shell=True",   True),
]


def solve(code: str) -> List[str]:
    """Return sorted, de-duplicated vulnerability tags for dangerous patterns in Python source."""
    if not code:
        return []
    findings = set()
    for tag, trigger, case_sensitive in RULES:
        haystack = code if case_sensitive else code.lower()
        needle = trigger if case_sensitive else trigger.lower()
        if needle in haystack:
            findings.add(tag)
    return sorted(findings)
```

1.5.c · AI suggesting detection rules

3 pts · auto-graded on the site

Coding problem, 1.5.c: Generate detection rules from a login log

Skill: Detect Attacks · **Maps to:** 1.5.A · **Language:** Python 3 (pytest)

Problem

A SOC receives a stream of authentication log events. Your task is to implement a brute-force detection engine: scan the log for source IPs that generate 5 or more `login_fail` events within any 60-second window, and emit a structured detection-rule string for each flagged IP.

This mirrors how an AI-assisted SIEM generates candidate detection rules from raw log data.

Function contract

Implement `solve(logs: list[dict]) -> list[str]` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`logs`, a list of dicts, each with:

- `ts` (int): Unix timestamp in seconds
- `src` (str): source IP address
- `event` (str): event type; only `"login_fail"` counts toward brute-force detection
- ok (bool): True = success, False = failure

Output

A **sorted** list of detection-rule strings. Format for each triggered IP:

`ALERT brute_force src=<IP> count=<N> window=60s`

where <N> is the maximum number of `login_fail` events from that IP found within any contiguous 60-second window (i.e., events where `max_ts - min_ts <= 59`). Rules are sorted lexicographically. At most one rule per IP.

Examples

```
logs = [
    {"ts": 0,  "src": "1.2.3.4", "event": "login_fail", "ok": False},
    {"ts": 10, "src": "1.2.3.4", "event": "login_fail", "ok": False},
    {"ts": 20, "src": "1.2.3.4", "event": "login_fail", "ok": False},
    {"ts": 30, "src": "1.2.3.4", "event": "login_fail", "ok": False},
    {"ts": 40, "src": "1.2.3.4", "event": "login_fail", "ok": False},
]
solve(logs) -> ["ALERT brute_force src=1.2.3.4 count=5 window=60s"]

logs = [
    {"ts": 0,  "src": "9.9.9.9", "event": "login_fail", "ok": False},
    {"ts": 10, "src": "9.9.9.9", "event": "login_fail", "ok": False},
]
solve(logs) -> []   # only 2 failures — below threshold

solve([]) -> []     # empty log
```

Algorithm (sliding window)

1. Collect all `login_fail` events and group by `src`. 2. Sort each group by `ts`. 3. For each group, slide a window: for each event at position `i`, count how many events starting at `i` have `ts <= events[i].ts + 59`. 4. If the maximum window count for any source is >= 5, emit its rule string. 5. Return the list sorted lexicographically.

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 1.5.c. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml).

solve(logs) -> sorted list of detection-rule strings.
Rule fires when an IP has >= 5 login_fail events within any 60-second window.
Rule string format: 'ALERT brute_force src=<IP> count=<N> window=60s'
"""
from typing import List, Dict, Any


THRESHOLD = 5
WINDOW_SECS = 59  # inclusive: events[i].ts + 59 covers a 60-second span


def solve(logs: List[Dict[str, Any]]) -> List[str]:
    """Return sorted detection-rule strings for IPs exceeding the brute-force threshold."""
    # Group login_fail timestamps by source IP
    failures: Dict[str, List[int]] = {}
    for entry in logs:
        if entry.get("event") == "login_fail":
            src = entry["src"]
            failures.setdefault(src, []).append(entry["ts"])

    rules = []
    for src, timestamps in failures.items():
        timestamps.sort()
        max_window = _max_window_count(timestamps)
        if max_window >= THRESHOLD:
            rules.append(f"ALERT brute_force src={src} count={max_window} window=60s")

    return sorted(rules)


def _max_window_count(timestamps: List[int]) -> int:
    """Return the maximum number of events within any 60-second window (sliding window)."""
    best = 0
    n = len(timestamps)
    right = 0
    for left in range(n):
        # advance right while timestamps[right] <= timestamps[left] + WINDOW_SECS
        while right < n and timestamps[right] <= timestamps[left] + WINDOW_SECS:
            right += 1
        count = right - left
        if count > best:
```

```
            best = count
    return best
```

1.5.d · Faster, more accurate threat detection and response

3 pts · auto-graded on the site

Coding problem, 1.5.d: Compute SOC incident-response metrics

Skill: Detect Attacks, Collaborate · **Maps to:** 1.5.B · **Language:** Python 3 (pytest)

Problem

A SOC dashboard needs two key metrics to measure how well AI-assisted response is performing:

1. **MTTR** (Mean Time to Respond): the average seconds from detection to resolution, computed across all incidents in the dataset. 2. **auto_resolved**: the count of incidents that were closed automatically by an AI playbook (no human analyst required).

Implement `solve(incidents)` that reads a list of incident records and returns both metrics.

Function contract

Implement `solve(incidents: list[dict]) -> dict` in `starter.py` (see `interface.md` / `blackbox.yaml`).

Input

`incidents` -- a list of dicts, each with:

- `id` (str): unique incident identifier
- `detected_at` (int): Unix timestamp (seconds) when detected
- `resolved_at` (int): Unix timestamp (seconds) when resolved; always > detected_at
- `severity` (str): one of `"critical"`, `"high"`, `"medium"`, `"low"`
- `auto` (bool): True if auto-resolved, False if human-resolved

Output

A dict with exactly two keys:

- `"mttr"` (float): mean response time in seconds across all incidents
- `"auto_resolved"` (int): count of incidents where `auto == True`

Formula: `mttr = sum(resolved_at - detected_at for each incident) / len(incidents)`

Return `{"mttr": 0.0, "auto_resolved": 0}` for empty input.

Examples

```
incidents = [
    {"id": "INC-001", "detected_at": 1000, "resolved_at": 1120, "severity": "critical", "auto": False},
    {"id": "INC-002", "detected_at": 2000, "resolved_at": 2080, "severity": "high",     "auto": True},
]
# response times: 120s, 80s -> mean = 100.0; auto_resolved = 1
solve(incidents) -> {"mttr": 100.0, "auto_resolved": 1}

solve([]) -> {"mttr": 0.0, "auto_resolved": 0}

incidents = [
    {"id": "INC-003", "detected_at": 0, "resolved_at": 300, "severity": "low", "auto": True},
```

```
]
solve(incidents) -> {"mttr": 300.0, "auto_resolved": 1}
```

Constraints

- Deterministic, offline, standard library only.
- Must pass all cases in `tests.py` (visible + hidden).

How to run

```
cd problem && python -m pytest tests.py -q
```

Reference solution:

```
"""Reference solution for 1.5.d. MUST pass 100% of tests.py.
Also the default 'blackbox' implementation students can swap out (see blackbox.yaml).

solve(incidents) -> {"mttr": float, "auto_resolved": int}
  mttr = mean(resolved_at - detected_at) across all incidents.
  auto_resolved = count of incidents with auto == True.
  Empty input returns {"mttr": 0.0, "auto_resolved": 0}.
"""
from typing import List, Dict, Any


def solve(incidents: List[Dict[str, Any]]) -> Dict[str, Any]:
    """Compute SOC MTTR and auto-resolved count from a list of incident records."""
    if not incidents:
        return {"mttr": 0.0, "auto_resolved": 0}

    total_time = sum(inc["resolved_at"] - inc["detected_at"] for inc in incidents)
    auto_count = sum(1 for inc in incidents if inc.get("auto") is True)

    return {
        "mttr": total_time / len(incidents),
        "auto_resolved": auto_count,
    }
```